

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards

(BL2,BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:25

Annexure A

Coding challenge

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Coding challenge

1. Write a Python script that generates a star topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.
2. Code a function ``validate_segment(length_m)`` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.
3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named ``csma_log.txt``.
4. Using Python's ``socket`` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.
5. Create a class ``StarSwitch`` with methods ``add_port(host_id)``, ``remove_port(host_id)``, and ``broadcast(frame)``. Broadcast must send a frame to all ports except the source and log each delivery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1. Write a Python script that generates a star-topology diagram (ASCII or graphical) given n hosts and a central switch. The script must print each host label, the switch name, and the connecting Cat6 UTP cable length.

```
# Star Topology using WHILE loop
```

```
n = int(input("Enter the number of hosts: "))
```

```
switch_name = input("Enter the switch name: ")
```

```
print("\nEnter Cat6 UTP cable length for each host:")
```

```
cables = []
```

```
i = 1
```

```
while i <= n:
```

```
    length = float(input(f"Length of cable for Host {i} (meters): "))
```

```
    cables.append(length)
```

```
    i += 1
```

```
print("\n===== STAR TOPOLOGY =====\n")
```

```
print("          " + switch_name)
```

```
print("          |")
```

```
print("      Central Switch")
```

```
print("          |")
```

```
i = 1
```

```
while i <= n:
```

```
    print(f"          |")
```

```
    print(f"          -- Cat6 ({cables[i-1]} m) --> Host {i}")
```

```
    i += 1
```

```
print("\n=====")
```

```
print("Topology: Star")
```

```
print("Switch :", switch_name)
```

```
print("Hosts  :", n)
```

```
print("Cable   : Cat6 UTP")
```

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-1.py ====
Enter the number of hosts: 4
Enter the switch name: SW1

Enter Cat6 UTP cable length for each host:
Length of cable for Host 1 (meters): 5
Length of cable for Host 2 (meters): 8
Length of cable for Host 3 (meters): 6
Length of cable for Host 4 (meters): 10

===== STAR TOPOLOGY =====
          SW1
          |
    Central Switch
          |
    -- Cat6 (5.0 m) --> Host 1
          |
    -- Cat6 (8.0 m) --> Host 2
          |
    -- Cat6 (6.0 m) --> Host 3
          |
    -- Cat6 (10.0 m) --> Host 4

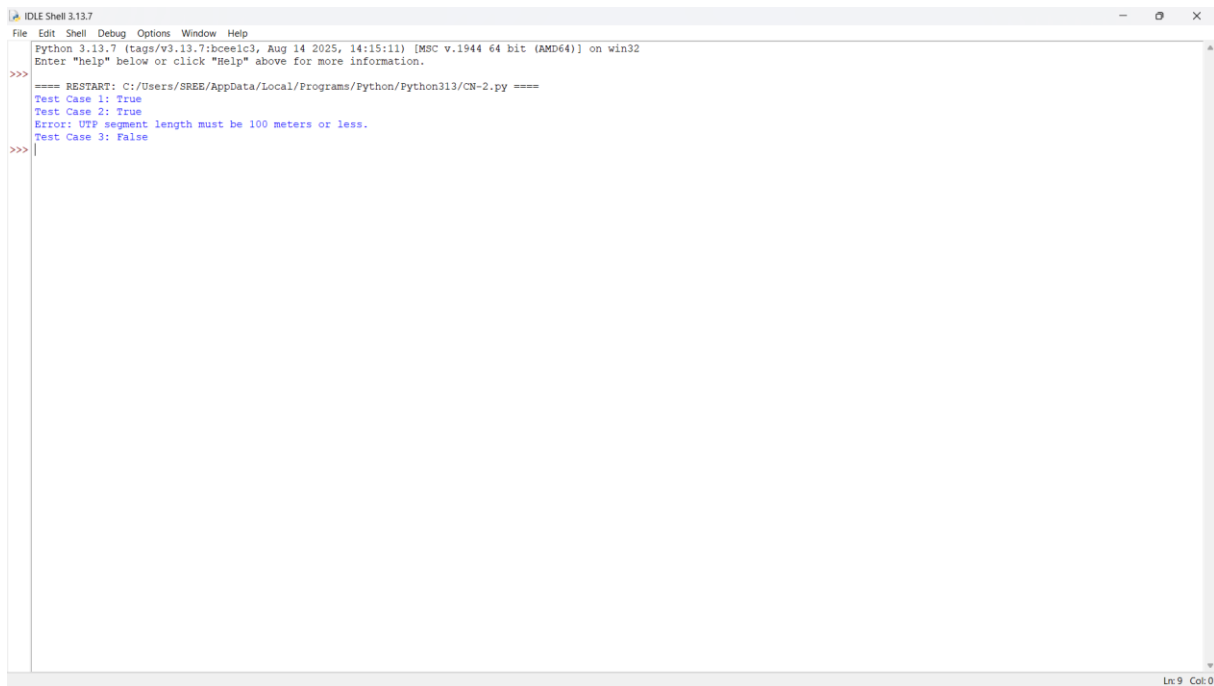
=====
Topology: Star
Switch   : SW1
Hosts    : 4
Cable    : Cat6 UTP
>>>
```

2. Code a function `validate_segment(length_m)` that returns True if the UTP segment is within IEEE 802.3 limits (≤ 100 m) and False with an error message otherwise. Include at least 3 test cases.

```
def validate_segment(length_m):
    if length_m <= 100:
        return True
    else:
        print("Error: UTP segment length must be 100 meters or less.")
        return False
```

Test cases

```
print("Test Case 1:", validate_segment(50))
print("Test Case 2:", validate_segment(100))
print("Test Case 3:", validate_segment(120))
```



```
Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-2.py ====
Test Case 1: True
Test Case 2: True
Error: UTF segment length must be 100 meters or less.
Test Case 3: False
>>>
```

3. Write a script that simulates CSMA/CD on a star topology with 4 hosts. Log every collision detection event and the back-off interval to a file named 'csma_log.txt'.

```
import random
import time
```

```
# Number of hosts
```

```
hosts = ["Host1", "Host2", "Host3", "Host4"]
```

```
# Open log file
```

```
log_file = open("csma_log.txt", "w")
```

```
log_file.write("CSMA/CD Simulation - Star Topology\n")
```

```
log_file.write("=====\n\n")
```

```
# Simulate 10 transmission attempts
```

```
for attempt in range(1, 11):
```

```
    # Select two hosts randomly
```

```
    host1, host2 = random.sample(hosts, 2)
```

```
    log_file.write(f"Transmission Attempt {attempt}\n")
```

```
    log_file.write(f"{host1} and {host2} are attempting to transmit.\n")
```

```
# Simulate collision
```

```
collision = random.choice([True, False])
```

```
if collision:
```

```
    log_file.write("Collision detected!\n")
```

```

# Generate random back-off interval
backoff1 = random.randint(1, 10)
backoff2 = random.randint(1, 10)

log_file.write(
    f"{host1} back-off interval: {backoff1} units\n"
)
log_file.write(
    f"{host2} back-off interval: {backoff2} units\n"
)

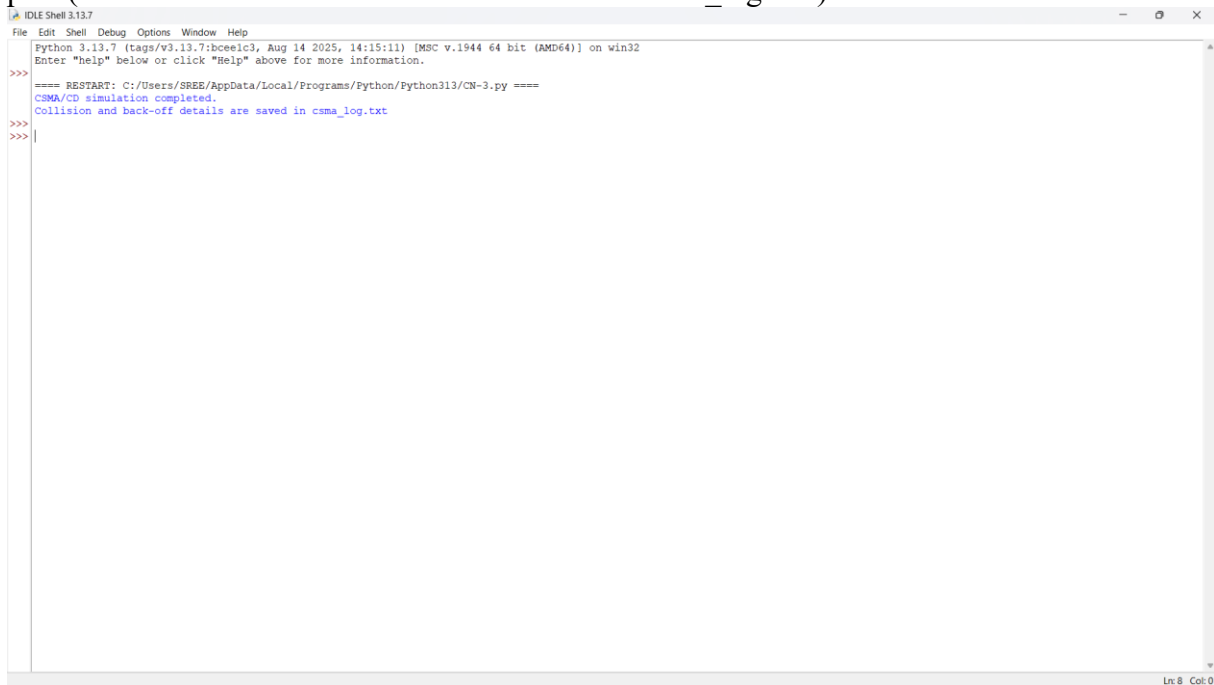
else:
    log_file.write("No collision. Transmission successful.\n")

log_file.write("-----\n")

log_file.close()

print("CSMA/CD simulation completed.")
print("Collision and back-off details are saved in csma_log.txt")

```



The screenshot shows a Python IDLE Shell window with the following content:

```

Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-3.py ====
CSMA/CD simulation completed.
Collision and back-off details are saved in csma_log.txt
>>>
>>>

```

The status bar at the bottom right indicates "Ln: 8 Col: 0".

4. Using Python's `socket` library, write a client-server program that models two hosts communicating through a star switch. Demonstrate message forwarding by printing the switch's MAC address table after each frame transmission.

```
import socket

# Switch MAC address
SWITCH_MAC = "AA:BB:CC:DD:EE:FF"

# MAC address table
mac_table = {}

# Create TCP socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 5000))
server.listen(2)

print("Star Switch is running...")
print("Switch MAC Address:", SWITCH_MAC)

# Accept two host connections
conn1, addr1 = server.accept()
print("Host 1 connected:", addr1)

conn2, addr2 = server.accept()
print("Host 2 connected:", addr2)

# Host MAC addresses
HOST1_MAC = "00:11:22:33:44:01"
HOST2_MAC = "00:11:22:33:44:02"

mac_table[HOST1_MAC] = "Port 1"
mac_table[HOST2_MAC] = "Port 2"

def show_mac_table():
    print("\nSwitch MAC Address Table")
    print("-----")

    for mac, port in mac_table.items():
        print(f'MAC Address: {mac} Port: {port}')

    print("-----")

# Receive message from Host 1
message = conn1.recv(1024).decode()

print("\nFrame received by switch:")
```



```

print("Source MAC:", HOST1_MAC)
print("Destination MAC:", HOST2_MAC)
print("Message:", message)

print("\nSwitch forwarding frame from Host 1 to Host 2...")
conn2.send(message.encode())

show_mac_table()

# Receive reply from Host 2
reply = conn2.recv(1024).decode()

print("\nFrame received by switch:")
print("Source MAC:", HOST2_MAC)
print("Destination MAC:", HOST1_MAC)
print("Message:", reply)

print("\nSwitch forwarding frame from Host 2 to Host 1...")
conn1.send(reply.encode())

show_mac_table()

conn1.close()
conn2.close()
server.close()

print("\nCommunication completed.")

```



```

Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-3.py =====
CSMA/CD simulation completed.
Collision and back-off details are saved in csma_log.txt
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-4.py =====
Star Switch is running...
Switch MAC Address: AA:BB:CC:DD:EE:FF
...
Star Switch is running...
...
Switch MAC Address: AA:BB:CC:DD:EE:FF
...
Host 1 connected: ('127.0.0.1', 52341)
...
Host 2 connected: ('127.0.0.1', 52342)
...
Frame received by switch:
...
Source MAC: 00:11:22:33:44:01
...
Destination MAC: 00:11:22:33:44:02
...
Message: Hello Host 2
...
Switch forwarding frame from Host 1 to Host 2...
...
Switch MAC Address Table
...
MAC Address: 00:11:22:33:44:01 Port: Port 1
MAC Address: 00:11:22:33:44:02 Port: Port 2
...
Frame received by switch:
...
Source MAC: 00:11:22:33:44:02
...
Destination MAC: 00:11:22:33:44:01
...
Message: Hello Host 1
...
Switch forwarding frame from Host 2 to Host 1...
...
Switch MAC Address Table
...
MAC Address: 00:11:22:33:44:01 Port: Port 1
MAC Address: 00:11:22:33:44:02 Port: Port 2
...
Communication completed.
...

```

5.Create a class `StarSwitch` with methods `add\_port(host\_id)`, `remove\_port(host\_id)`, and `broadcast(frame)`. Broadcast must send a frame to all ports except the source and log each delivery.

```
class StarSwitch:
    def __init__(self):
        # Store connected hosts
        self.ports = {}

    def add_port(self, host_id):
        """Add a host to the switch."""
        if host_id not in self.ports:
            self.ports[host_id] = True
            print(f'{host_id} connected to the switch.')
        else:
            print(f'{host_id} is already connected.')

    def remove_port(self, host_id):
        """Remove a host from the switch."""
        if host_id in self.ports:
            del self.ports[host_id]
            print(f'{host_id} disconnected from the switch.')
        else:
            print(f'{host_id} not found.')

    def broadcast(self, frame):
        """Send frame to all ports except the source."""
        source = frame["source"]
        message = frame["message"]

        print("\nBroadcasting frame...")
        print("Source :", source)
        print("Message:", message)

        for host_id in self.ports:
            if host_id != source:
                print(f'Frame delivered to {host_id}')
```

```
# Create a StarSwitch object
```

```
switch = StarSwitch()
```

```
# Add hosts
```

```
switch.add_port("Host1")
```

```
switch.add_port("Host2")
```

```
switch.add_port("Host3")
```

```
switch.add_port("Host4")
```

```
# Create a frame
```

```
frame = {
```

```
    "source": "Host1",
```

```
    "message": "Hello from Host1"
```

```
}
```

```
# Broadcast the frame
```

```
switch.broadcast(frame)
```

```
# Remove a host
```

```
switch.remove_port("Host4")
```

```
# Broadcast another frame
```

```
frame2 = {
```

```
    "source": "Host2",
```

```
    "message": "Hello from Host2"
```

```
}
```

```
switch.broadcast(frame2)
```

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bce1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
==== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-3.py ====
CSMA/CD simulation completed.
Collision and back-off details are saved in csma_log.txt
>>>
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-4.py =====
Star Switch is running...
Switch MAC Address: AA:BB:CC:DD:EE:FF
===== RESTART: C:/Users/SREE/AppData/Local/Programs/Python/Python313/CN-5.py =====
Host1 connected to the switch.
Host2 connected to the switch.
Host3 connected to the switch.
Host4 connected to the switch.

Broadcasting frame...
Source : Host1
Message: Hello from Host1
Frame delivered to Host2
Frame delivered to Host3
Frame delivered to Host4
Host4 disconnected from the switch.

Broadcasting frame...
Source : Host2
Message: Hello from Host2
Frame delivered to Host1
Frame delivered to Host3
>>>
```

Ln: 32 Col: 0